

Αναλυτικός Οδηγός Κώδικα (Σκονάκι Παρουσίασης)

Αυτό το έγγραφο αναλύει τη δομή και τη λειτουργικότητα κάθε αρχείου του πηγαίου κώδικα, επεξηγώντας με ακρίβεια τον ρόλο του (Τι κάνει) και τον τεχνικό του μηχανισμό (ΠΩΣ το κάνει). Ιδανικό για γρήγορη αναφορά κατά τη διάρκεια των ερωτήσεων της παρουσίασης.

1. Κεντρική Εκκίνηση & Αρχιτεκτονική (Root)

Main.java

Τι κάνει: Είναι το κεντρικό σημείο εκκίνησης (Composition Root) ολόκληρης της εφαρμογής, όπου αρχικοποιούνται και συνδέονται όλα τα υποσυστήματα.

Πώς το κάνει: Δημιουργεί το UI της κονσόλας, αρχικοποιεί τον `JsonGameLoader` για να γεμίσει τη μνήμη, κατασκευάζει τα αντικείμενα `PlayerImpl` και `GameState`, εισάγει όλες τις διαθέσιμες εντολές στο `CommandRegistry` (Register) και τελικά εκκινεί τη μηχανή καλώντας τη `GameEngine.start()`.

App.java

Τι κάνει / Πώς: Πρόκειται για ένα boilerplate αρχείο ("Hello World") που δημιουργήθηκε αυτόματα κατά το setup του project. Δεν συμμετέχει σε καμία λειτουργία της αρχιτεκτονικής μας και αγνοείται πλήρως.

2. Ο Πυρήνας του Παιχνιδιού (Package: engine)

GameEngine.java

Τι κάνει: Διαχειρίζεται τον κεντρικό βρόχο (Game Loop) του παιχνιδιού και το συγχρονισμό εισόδου-εξόδου.

Πώς το κάνει: Εκτελεί μια λούπα `while (!gameState.isGameOver())`. Σε κάθε επανάληψη, λαμβάνει το string εισόδου από το `ui.getUserInput()`, το προωθεί στον `CommandDispatcher` για εκτέλεση και στο τέλος καλεί τη μέθοδο `evaluateGameConditions()` για να ελέγξει αν ικανοποιούνται τα flags νίκης ή ήττας ώστε να τερματίσει το loop.

GameState.java

Τι κάνει: Λειτουργεί ως η κεντρική, ενιαία "μνήμη" της τρέχουσας συνεδρίας (session state).

Πώς το κάνει: Κρατάει pointers για τον ζωντανό Player και το RoomRegistry. Αποθηκεύει strings για τη δυσκολία (`difficultyLevel`) και τον στόχο (`currentGoal`), καθώς και μια `List` για το ιστορικό των εντολών. Χρησιμοποιεί ένα `Map` για flags καταστάσεων (π.χ. αν ο παίκτης νίκησε).

ConditionSystem.java

Τι κάνει / Πώς: Βοηθητικό υποσύστημα ελέγχου κανόνων. Αυτή τη στιγμή υλοποιεί τη μέθοδο `canUseItemOnTarget`, η οποία ελέγχει αν ο παίκτης κατέχει ένα αντικείμενο καλώντας την `player.hasItem()`, επιστρέφοντας true ή false.

3. Σύστημα Φόρτωσης Δεδομένων (Package: loader)

GameLoader.java (Interface)

Τι κάνει / Πώς: Ορίζει το γενικό συμβόλαιο φόρτωσης βάσει του Strategy Pattern, δηλώνοντας την υπογραφή της μεθόδου `Room loadGame(String filePath)`.

JsonGameLoader.java

Τι κάνει: Αναλαμβάνει το parsing του εξωτερικού αρχείου σεναρίου και τη μετατροπή του σε αντικείμενα Java.

Πώς το κάνει: Χρησιμοποιεί τη βιβλιοθήκη **Gson** της Google. Διαβάζει το αρχείο, το μετατρέπει σε DTO κλάσεις, κάνει loop στα δεδομένα και δημιουργεί instances των ``RoomImpl``, ``ItemImpl``, ``ExitImpl`` και ``NpcImpl``. Συνδέει τις οντότητες μεταξύ τους, τις καταχωρεί στο ``RoomRegistry`` και επιστρέφει το αρχικό δωμάτιο εκκίνησης.

GameDataDTO.java

Τι κάνει / Πώς: Μια καθαρή κλάση μεταφοράς δεδομένων (Data Transfer Object) χωρίς επιχειρηματική λογική. Περιέχει εσωτερικές static κλάσεις (`RoomDTO`, `ItemDTO`, κ.λπ.) με fields που ονομάζονται **ακριβώς όπως τα keys του JSON**, επιτρέποντας στο Gson να κάνει reflection και αυτόματο mapping.

RoomRegistry.java

Τι κάνει / Πώς: Μια in-memory βάση δεδομένων για τα δωμάτια. Χρησιμοποιεί ένα ``HashMap``, όπου το key είναι το μοναδικό ID του δωματίου (String) και το value είναι το instance του δωματίου. Επιτρέπει ακαριαία εύρεση και μετακίνηση με πολυπλοκότητα $O(1)$.

StubGameLoader.java

Τι κάνει / Πώς: Μια "ψεύτικη" (mock) υλοποίηση του loader που χρησιμοποιήθηκε στα πρώτα στάδια ανάπτυξης για τη δοκιμή της ροής πριν αναπτυχθεί το σύστημα JSON. Επιστρέφει μόνιμα null.

4. Μοντέλα Οντοτήτων (Package: model)

Room / RoomImpl.java

Τι κάνει / Πώς: Αντιπροσωπεύει μια τοποθεσία. Κρατάει strings για ID, όνομα, περιγραφή και διαχειρίζεται τρεις ``ArrayList`` συλλογές για τα ``Item``, ``Exit`` και ``Npc`` που βρίσκονται μέσα στο δωμάτιο, παρέχοντας μεθόδους προσθήκης, αφαίρεσης και αναζήτησης.

Player / PlayerImpl.java

Τι κάνει / Πώς: Κρατάει την κατάσταση του χρήστη. Έχει μια αναφορά στο τρέχον δωμάτιο (``Room currentRoom``) και μια ``ArrayList`` που λειτουργεί ως το inventory του παίκτη.

Npc / NpcImpl.java

Τι κάνει / Πώς: Υλοποιεί τους χαρακτήρες του παιχνιδιού ως Finite State Machines (Μηχανές Καταστάσεων). Κρατάει το ``startingState`` / ``currentState`` και ένα ``Map`` (dialogs). Όταν καλείται η ``talk()``, επιστρέφει το κείμενο που αντιστοιχεί στο τρέχον state από το Map, ή ένα default μήνυμα σιωπής αν το state δεν βρεθεί.

Item / ItemImpl.java & Exit / ExitImpl.java

Τι κάνει / Πώς: Το Item είναι μια απλή κλάση με όνομα και περιγραφή. Το Exit αναπαριστά τις διόδους: κρατάει την κατεύθυνση, το ID του δωματίου προορισμού, ένα boolean ``isLocked`` και το String ``requiredItemId`` που δηλώνει ποιο αντικείμενο ξεκλειδώνει τη συγκεκριμένη έξοδο.

5. Μηχανισμός Εντολών - Command Pattern (Package: command)

Command, CommandDispatcher & CommandDispatcherImpl.java

Τι κάνει / Πώς: Εφαρμόζουν το Command Pattern. Το Interface ``Command`` ορίζει τη μέθοδο ``execute()``. Ο Dispatcher λαμβάνει το ακατέργαστο input, καλεί τον ``CommandParser`` για ανάλυση, καταγράφει την εντολή στο ιστορικό του ``GameState``, αναζητά το Command αντικείμενο από το ``CommandRegistry`` και εκτελεί την ``execute()`` πάνω του.

CommandParser & ParsedCommand.java

Τι κάνει / Πώς: Ο Parser σπάει το String εισόδου σε tokens με ``split("\\s+")``. Αφαιρεί τις λέξεις που ανήκουν στη λίστα ``stopWords`` ("the", "a"), αντικαθιστά τα ρήματα με τις κύριες εκδοχές τους μέσω του ``SynonymMap``, εντοπίζει προθέσεις ("to", "on") και επιστρέφει ένα αμετάβλητο (immutable) αντικείμενο ``ParsedCommand`` γεμάτο με τα καθαρισμένα ορίσματα.

SynonymMap.java & CommandRegistry.java

Τι κάνει / Πώς: Το ``SynonymMap`` διαβάζει το ``synonyms.txt`` κατά την εκκίνηση, κάνει split με βάση το χαρακτήρα ``:`` και ``,` και γεμίζει ένα ``HashMap`` που χαρτογραφεί κάθε συνώνυμο (π.χ. "grab") στην αρχική εντολή ("take"). Το ``CommandRegistry`` είναι ένα ``HashMap`` που αντιστοιχίζει το τελικό ρήμα-εντολή με το αντίστοιχο instance κλάσης.

6. Συγκεκριμένες Εντολές (Concrete Commands)

Όλες οι κλάσεις εντολών υλοποιούν το ``Command`` interface και εκτελούν τη δική τους απομονωμένη λογική:

- **GoCommand:** Ελέγχει αν υπάρχει η έξοδος στο δωμάτιο. Αν ``isLocked``, αλλάζει το room του παίκτη. Αν το ID του νέου δωματίου είναι "outside", θέτει το flag νίκης σε true.
- **LookCommand:** Καλεί τις μεθόδους εκτύπωσης του UI για να δείξει το όνομα, την περιγραφή του δωματίου, τις εξόδους και τα items στο πάτωμα.
- **TakeCommand / DropCommand:** Μεταφέρουν αντικείμενα. Η Take καλεί ``room.removeItem()`` και ``player.addItem()``, ενώ η Drop κάνει το αντίστροφο.
- **UseCommand (SMART): Έξυπνη Λειτουργία!** Ελέγχει αν το item υπάρχει στο inventory του παίκτη. Στη συνέχεια, κάνει αυτόματα loop σε όλες τις εξόδους (``Exit``) του τρέχοντος δωματίου. Αν βρει μια έξοδο που είναι κλειδωμένη (``isLocked()``) και το ``requiredItemId`` της ταιριάζει με το αντικείμενο, την ξεκλειδώνει αυτόματα, χωρίς να απαιτείται από τον παίκτη να γράψει την κατεύθυνση!
- **GiveCommand:** Ελέγχει αν ο NPC βρίσκεται στο δωμάτιο και αν το αντικείμενο ταιριάζει με το ``requiredItemId`` του NPC. Αν ναι, αφαιρεί το item από τον παίκτη, αλλάζει το state του NPC σε "happy" και ξεκλειδώνει αυτόματα όλες τις κλειδωμένες πόρτες του δωματίου.
- **TalkCommand:** Εντοπίζει τον NPC στο δωμάτιο και τυπώνει το string διαλόγου που επιστρέφει η μέθοδος ``npc.talk()``.
- **SaveCommand / LoadCommand:** Χρησιμοποιούν ``FileWriter`` και ``BufferedReader`` για να γράψουν/ διαβάσουν σε ένα απλό αρχείο κειμένου (``savegame.txt``) την κατάσταση (ID τρέχοντος δωματίου και επίπεδο δυσκολίας).
- **History, State, Difficulty, Quit:** Διαβάζουν ή τροποποιούν τις αντίστοιχες μεταβλητές/flags μέσα στο ``GameState`` και τυπώνουν τα αποτελέσματα μέσω του UI.

7. Διεπαφή Χρήστη (Package: ui)

ConsoleUI.java

Τι κάνει / Πώς: Απομονώνει την είσοδο/έξοδο. Χρησιμοποιεί έναν ``Scanner(System.in)`` για να διαβάζει γραμμές κειμένου από το πληκτρολόγιο και μεθόδους όπως η ``printMessage`` που εκτελούν ``System.out.println``. Διασφαλίζει ότι ο engine δεν έχει άμεση εξάρτηση από την κονσόλα (επιτρέποντας μελλοντική εύκολη μετάβαση σε GUI).